

软件过程与管理课程作业

刘瑞雪 2022141090116

一、项目软件过程定义

项目 HomePick 是一个租房点评社区，用户可以发布房源测评、收藏心仪房源、参与租房补贴秒杀。项目周期为 2025 年 8 月至 2025 年 11 月，技术栈包括 Spring Boot、MySQL、Redis、RocketMQ。项目的核心难点集中在缓存治理和高并发秒杀两个方向。

根据 ISO/IEC 12207 的过程分类思想，将项目中的活动划分为技术过程、支持过程和组织过程三类，具体定义如下。

（一）技术过程

需求分析阶段：结合租房场景确定了房源测评、房源收藏、秒杀抢购三个核心功能模块。

软件设计阶段：确定 B/S 架构，选定 Spring Boot 作为后端框架、MySQL 作为持久化数据库、Redis 作为缓存和秒杀核心组件、RocketMQ 作为异步消息队列。

编码实现阶段：主要工作集中在以下三个方面：

1. 使用 Redis 进行房源信息缓存，通过空对象缓存解决穿透问题、TTL 随机值解决雪崩问题、互斥锁加逻辑过期解决击穿问题。
2. 使用 Redisson 分布式锁实现一人一单的秒杀限制，基于 Redis 加 Lua 脚本保证库存扣减的原子性。
3. 引入 RocketMQ 将秒杀资格预检后的下单请求异步写入消息队列，实现流量削峰。

（二）支持过程

配置管理：全程使用 Git 进行版本控制，采用 Feature 分支模型，代码托管在 Gitee 上。

验证过程：使用 Postman 进行 API 功能测试，使用 JMeter 对秒杀核心链路进行压力测试。

（三）组织过程

项目阶段规划：项目周期被划分为四个阶段：基础功能搭建、缓存层实现、秒杀功能实现、异步优化，按阶段推进和跟踪。

质量保证：使用 SonarQube 进行静态代码扫描，确保代码规范。

二、软件过程定义的来源

上述软件过程定义主要依据 ISO/IEC 12207:2017《软件工程—软件生存周期过程》标准。该标准给出了软件生存周期中应当包含的全部过程和活动，包括技术过程、支持过程和组织过程三大类。按照这一分类框架，结合 HomePick 项目的实际技术活动和管理活动，逐一归入对应类别，形成了上述过程定义。同时，在剪裁过程中也参考了 ISO/IEC 15504 和 33001 的过程参考模型思想，但以 12207 作为主要裁剪基准。

三、过程的剪裁想法

ISO/IEC 12207 是为通用软件项目设计的，包含 30 余个详细过程和活动。对于一个为期三个月、仅一人的学术课程项目而言，直接照搬全部过程会带来极高的管理开销。因此，根据项目规模、周期、性质进行系统性剪裁。下面采用“过程剪裁表”的形式，逐类说明每个标准过程的处理方式及理由。

表 1 剪裁总表

标准过程（依据 ISO/IEC 12207）	剪裁方式	理由与具体调整
获取过程	删除	无外部供应商，所有代码及组件均为自主开发或开源依赖，不涉及采购合同。
供应过程	删除	无对外交付产品的商业合同，仅提交课程作业报告，无需向客户交付。
开发过程（技术过程）	保留并强化	核心需求、设计、实现、测试均完整执行；针对缓存治理和秒杀难点额外增加 Lua 脚本、分布式锁、RocketMQ 削峰等强化措施。
维护过程	删除	项目生命周期终点明确（学期末），无需考虑长期的系统维护、移植或退役。
文档过程	简化	取消正式文档计划，仅保留必要的代码注释及本作业报告，不编写独立用户手册或设计文档。
验证过程	保留并强化	除常规 API 测试外，针对秒杀高并发风险额外引入 JMeter 压力测试，验证系统吞吐量与数据一致性。
确认过程	简化	无外部客户验收，以开发人员自测及作业演示代替正式确认。
联合评审过程	删除	无多方利益相关者，不需要联合评审。
审计过程	删除	个人项目无独立第三方审计角色，相关质量保证通过 SonarQube 静态扫描及代码规范自查覆盖。
问题解决过程	简化	使用 GitHub Issues 记录 Bug 和待办项，未建立正式的问题管理数据库。
配置管理过程	保留	使用 Git + Gitee 进行版本控制，采用 Feature 分支模型，满

标准过程（依据 ISO/IEC 12207）	剪裁方式	理由与具体调整
		足配置项标识、版本控制的基本要求。
基础设施过程	简化	仅配置本地开发环境（IntelliJ IDEA, MySQL, Redis, RocketMQ），未搭建持续集成服务器或集群环境。
人力资源过程	删除	单人项目，无需技能培训、人员调度或绩效评估。
质量保证过程	替换/简化	取消正式 QA 计划及评审，改用 SonarQube 自动静态分析+代码自查替代，确保基本编码规范。
过程改进过程	删除	项目周期短，无度量数据积累与过程改进迭代需求。

剪裁的核心思想

基于本项目“个人、短期、学术”的特性，遵循以下原则进行剪裁：

- 1. 风险导向：保留并强化与核心风险直接相关的技术过程和验证过程。
- 2. 轻量管理：删除所有面向合同、团队协作、长期维护的重型管理过程。
- 3. 工具替代：用自动化工具（SonarQube、JMeter、Git）替代正式评审、部分验证和配置管理工作，降低管理成本。
- 4. 文档最小化：仅保留必要的代码注释和课程作业报告，取消正式文档过程。

通过上述剪裁，最终形成一个精简但核心环节完备的软件过程，避免了不必要的管理开销，同时保证了对缓存穿透/雪崩/击穿、秒杀原子性、流量削峰等关键技术的有效开发和验证。